Journal of Computational and Applied Mathematics 9 (1983) 177–191
North-Holland

Algorithm 25

Starting step size for an ODE solver *

H.A. WATTS
Sandia National Laboratories, Albuquerque, NM 87185, USA

Received 5 August 1982

Abstract: One of the more critical issues in solving ordinary differential equations by a step-by-step process occurs in the starting phase. Somehow the procedure must be supplied with an initial step size that is on scale for the problem at hand. It must be small enough to yield a reliable solution by the process, but not so small as to significantly affect the efficiency of solution. In this paper, we discuss an algorithm for obtaining a good starting step size and present a subroutine which can be readily used in most ODE solvers.

Keywords: Starting step size, ODES, local Lipschitz constant.

1. Introduction

One of the more critical issues in solving ordinary differential equations by a step-by-step process occurs in the starting phase. Somehow the procedure must be supplied with an initial step size that is on scale for the problem at hand. It must be small enough to yield a reliable solution by the process, but it should not be so small as to significantly affect the efficiency of solution. The more important of these two possibilities is obviously the reliability question. The first step taken by the code must reflect how fast the solution changes near the initial point. For general purpose computing, an automatic step size adjustment procedure for choosing subsequent steps is essential to produce an accurate solution efficiently. This step size control is usually based on estimates of the local errors incurred by the numerical method. Because most codes also employ algorithmic devices which restrict the step size control to be moderately varying (for reliability), subsequent steps usually tend to stay on scale of the problem. This is not always so, as sometimes happens when working with crude tolerances on problems having rapidly varying components. Nevertheless, most step size adjustment procedures deal reasonably well with all but the most abrupt changes, leaving the most serious danger confined to the starting step size.

In existing codes, several possibilities for obtaining an initial step size can be found. In many cases the user is asked to provide a guess and, more often than not, a somewhat arbitrary value is supplied. This is at least an annoyance to the user who, typically, has little idea of what a good value might be for his particular problem and for the method to be used in the code of his choice. Another possibility seen in practice is based on using some (fixed) fraction of the first output interval length. Since an acceptable starting step size depends on the initial behavior of the solution, it seems inappropriate for the choice to depend solely on the length of the interval of integration. For one thing, it does not take into account the accuracy tolerance requested. For another, the first output interval length may be set too large by users who have no interest in seeing the initial behavior of the solution. A starting step size that is much too large not only can cause some local phenomena to be missed entirely, but may instead result in a long sequence of wasted failed steps before the asymptotic theory becomes valid and the error estimator produces a good number to be used in the step size algorithm. In the prologs of all the codes in DEPAC [1], the user is told that the first step taken will not go beyond the first designated output point. Thus the user could, when necessary, restrict the length of the first step size in these codes. This is one way which the user can provide information about the initial scale of his problem.

* This work was supported by the U.S. Department of Energy.

To be certain that the starting step size is chosen small enough, Sedgwick [2] started every integration with the smallest permissible step size in the precision of the machine being used. While this approach has some merits, it is generally inefficient to work up to an appropriate value because of step size controls which may not allow too rapid of a change. In addition, the asymptotic error estimates are likely to be contaminated with roundoff errors to the extent of impairing a sufficiently rapid increase in step size. Another drawback in using this technique is that the effects of achieving excessive accuracy in the starting phase, due to using step sizes much smaller than necessary, will show up in the global errors obtained. This is important when the software is to be designed with the goal of obtaining (hopefully) a computed solution whose true accuracy has a rather uniform behavior as the tolerances are reduced and is actually comparable to the error tolerances for most problems.

Some time ago Shampine [3] suggested a rule of thumb choice for automatic selection of an initial step size h. It is derived from the assumption that the error made by a method of order p is h^p times the error made by a method of order zero, i.e. a constant approximation. For the differential equation problem $y' = f(x, y)$, $y(a) = \eta$, and the error tolerance parameter ε, this results in the starting step size being chosen from $\|h^p(hf(a, \eta))\| = \|h^{p+1}f(a, \eta)\| = \varepsilon$. This approach has been used in the Runge–Kutta code RKF45 [4,5] and in the Adams code STEP [6] for some years with fair success. However, from time to time, we have received feedback about troubles which appeared to arise from an inappropriate starting step size obtained with the device. An obvious defect is seen to occur when the initial slopes vanish. With lack of further information in this case, these codes use the designated interval length for the starting step size. This has occasionally led to difficulties for reasons which we have already mentioned. Even when the initial derivatives are nonzero, the above scheme does not always obtain a good starting step size that is indicative of how rapidly the solution may change locally.

We must insist that the start be performed with a suitable step size if we are to enhance the reliability of ODE solvers. The starting step size should take into account such things as the numerical method to be used initially, the requested error tolerances, and the local behavior of the differential equation being solved. Since we do not expect the user to typically be in a position to provide us with enough information, the code must undertake a more involved effort of examining the problem automatically. We believe that this is important enough so as not to mind doing a moderate amount of additional work.

For several years a group of workers at the University of Toronto, led by Hull and Enright, have advocated the use of a scale parameter to define the maximum appropriate step size for the entire integration. In DVERK [7], the user is asked to provide (as optional input) a rough measure of the Lipschitz constant for the problem. This value is then used to restrict the step size. In the comparison study [8] the authors chose an initial step size $h_0 = 1/|\lambda_{\max}|$, where $\lambda_{\max}$ is the eigenvalue of largest modulus of the Jacobian matrix for the system of differential equations. Shampine [9,10] discusses some alternatives for overcoming the defects of the earlier mentioned initial step size selection scheme. In [11] he continues his work aimed at making ODE solvers more reliable and suggests that codes compute estimates of local Lipschitz constants. With this additional information it was recommended, among other things, that use be made for controlling the step size choice. In his IMPEX2 code, Lindberg [12] computes an estimate of the Lipschitz constant and uses it in determining the starting step size. Recently, Gear [13,14] has examined techniques for improving the efficiency of the start-up phase for multistep methods and he also is concerned with selection of an appropriate initial step size. In this paper we combine and expand on some of the above ideas for the purpose of obtaining a good starting step size. We discuss an algorithm and present a subroutine which can be readily used in most ODE solvers (in some cases minor adaptations may be desirable).

2. Starting step size formula

For the initial value problem $y'(x) = f(x, y)$, $y(a) = \eta$, the Taylor expansion about the initial point results in:

$$\begin{aligned} y(a+h) &= y(a) + hy'(a) + \tfrac{1}{2}h^2y''(a) + \mathrm{O}(h^3) \\ &= y(a) + hf(a, \eta) \\ &\quad + \tfrac{1}{2}h^2\left\{\frac{\partial f}{\partial x} + \frac{\partial f}{\partial y}f\right\}_{(a,\eta)} + \mathrm{O}(h^3). \end{aligned}$$

The second derivative term $\partial f/\partial x + (\partial f/\partial y)f$ is evaluated at (a, η). Our approach to computing a better starting step size includes the effects of an approximation to this term.

Earlier we mentioned the rule of thumb assumption about the error of a method of order p being of the form $h^p E$, where $E \doteq \|hy'\|$ represented the error of a constant approximation. An equally valid assumption is that the error for a method of order p is of the form E^{p+1}. One reason for preferring the latter assumption is that it leads to a scale invariant scheme for step size determination. In particular, changing the independent variable to $t = \alpha x$ leads to a correctly scaled starting step of αh. The basic approach which we shall adopt has $E \doteq \frac{1}{2}h^2\|y''\|$ for the error in the Euler approximation, and the assumption we make is that the error for a method of order p is of the form $E^{(p+1)/2}$. Thus for a given tolerance parameter ε, the length of a suitable step size can be obtained from

$$h = \frac{\sqrt{2}\,\varepsilon^{1/p+1}}{\sqrt{\|y''(a)\|}}.$$

Approximations to the second derivative can be defined by numerical differences

$$y''(x) \doteq \frac{y(x+\delta x) - 2y(x) + y(x-\delta x)}{(\delta x)^2},$$

or

$$y''(x) \doteq \frac{1}{\delta x}\{f(x+\delta x, y(x+\delta x)) - f(x, y(x))\}.$$

However, we have not given any serious considerations to computing such approximations for several reasons. Shampine [9] has pointed out that to compute difference ratios like these, we must already have an idea of the proper scale of the problem in order to choose appropriate perturbations δx. In the above difference expressions it is also necessary to obtain reasonable approximations for $y(x \pm \delta x)$. Of course Euler's approximation is a natural choice, and another might be to allow perturbations within the error band provided by the user supplied error tolerances. Taking all of these facts together, this type of second derivative estimate is likely to be more sensitive than an alternative. In addition, we wanted an estimate of the local Lipschitz constant. Thus we actually compute a bound for the second derivative term $\|y''\| \leqslant \|\partial f/\partial x\| + \|\partial f/\partial y\|\|f\|$, providing estimates of the variations of the equations with respect to both independent and dependent variables separately.

3. Numerical differencing for estimating derivatives

Consider the problem of estimating the derivative of a scalar function $f(z)$ with respect to the variable z. Computational difficulties associated with using difference approximations are well known. Choosing too large a perturbation increment in the variable is likely to cause an unacceptably large truncation error, whereas choosing too small of an increment can lead to severe cancellation and roundoff error contamination. Sometimes user-supplied error tolerances are used in defining the perturbation increments. On the one hand, this may sound perfectly reasonable since a computed approximation is allowed to have an error which is as big as the tolerances provided. Thus it is argued that a corresponding perturbation would be acceptable. We do not think this is a good idea in general but rather that it should, in some way, depend on the machine precision. One possibility which works well most of the time is to taken an increment $\delta z = \sqrt{u}\ z$, where u is a small machine-dependent number which defines the relative precision of the computations (often referred to as the computer unit roundoff error or machine epsilon). For our purpose we are not interested in obtaining highly accurate approximations to derivatives, just reasonably good estimates. Thus we lean more to choosing an appropriate perturbation increment aimed at avoiding serious cancellation and rounding effects. This motivates the choice of $u^{3/8}$ as opposed to using $\sqrt{u}$ [22].

The problem of estimating derivatives of a vector function, $f(v)$, with respect to the various components of a vector v, becomes a harder task. This is because f is usually provided as a vector from a subroutine rather than computing and returning each individual equation (function component) separately. An appropriate perturbation increment for a particular variable and a certain equation may not be appropriate for a different equation. Scaling difficulties usually accompany equations having a wide range of magnitudes between the variables, commonly occurring with stiff differential equations. There seems to be little that

can be done to circumvent difficult scaling problems where roundoff errors are made prominent by catastrophic cancellation, at least when the aim is to address general nonlinear systems and the effort of estimating derivatives is to be kept at a modest level of cost. Curtis and Reid [15], and Stepleman and Winarsky [16], and Oliver [17] propose schemes for increasing the reliability of derivative approximations, but we consider them to be too expensive for our purposes.

4. Estimating $\|\partial f/\partial x\|$

For the task of estimating $\|\partial f/\partial x\|$ at the initial point a, we shall utilize an additional value b, of the independent variable, which defines the direction of integration. Typically, b can be thought of as the first output point of interest, which will be used in defining the perturbation increment δa. (We shall presume that $b \neq a$ on the machine being used.) In particular, we consider it important to make the change occur in the direction that the integration will proceed. This is because the differential equation may not even be defined on the other side of the initial point or it may be discontinuous there. Furthermore, it seems reasonable in general to restrict $|\delta a|$ so as not to be larger than $|b-a|$. However, in doing this we must also take precaution against a bad choice of b which would result in $|\delta a|$ being too small relative to a in the machine precision available. For this protection we shall insist that $|\delta a| \geq 100u|a|$. Taken together this amounts to choosing the change in a by

$$\delta a = \max\{\min(u^{3/8}|a|, |b-a|), 100u|a|\}\operatorname{sgn}(b-a).$$

If this underflows to zero, we just set $\delta a = u^{3/8}\cdot(b-a)$. Finally, we compute an estimate of $\|\partial f/\partial x\|$ at the initial point from

$$\left\|\frac{\partial f}{\partial x}\right\| \doteq \frac{\|f(a+\delta a, y(a)) - f(a, y(a))\|}{|\delta a|}.$$

5. Estimating a local Lipschitz constant

Assuming that f has continuous first partial derivatives in a neighborhood of $(a, y(a))$, the mean value theorem states that

$$f(a, y+\delta y) - f(a, y) = (\partial f/\partial y)\delta y,$$

where the Jacobian matrix $J = \delta f/\delta y$ has entries evaluated at points along the line between y and $y+\delta y$. When we refer to a local Lipschitz constant, we mean a constant L such that

$$\|f(a, y+\delta y) - f(a, y)\| \leq L\|\delta y\|$$

holds in a small neighborhood of $(a, y(a))$. Thus we shall suppose that $L \doteq \|J\|$ represents a good approximation locally which we shall estimate be forming

$$\frac{\|f(a, y+\delta y) - f(a, y)\|}{\|\delta y\|}.$$

This represents a measure of the derivative of f in the direction of δy. We would like to find directions in which f changes rapidly and, furthermore, we would like to accomplish this in just a few evaluations of f.

Lindberg [12] and Shampine [11] both describe an analog to the power method for estimating dominant eigenvalues. This approach defines a sequence of perturbed values

$$y^{(i)} = y^{(0)} + \delta y^{(i)}, \quad i = 1, 2, \ldots$$

along with

$$\rho_i = \frac{\|f^{(i)} - f^{(0)}\|}{\|\delta y^{(i)}\|}$$

and

$$\delta y^{(i+1)} = \frac{1}{\rho_i}\left[f^{(i)} - f^{(0)}\right] \doteq \frac{1}{\rho_i} J\delta y^{(i)}$$

$$\doteq \frac{1}{\rho_i\rho_{i-1}\cdots\rho_1} J^i \delta y^{(1)}.$$

In the above process, $y^{(0)}$ and $y^{(1)}$ (or $\delta y^{(1)}$) must be chosen in some other way to get the scheme started. Also, $f^{(i)}$ denotes $f(a, y^{(i)})$ and $\|\delta y^{(i)}\|$ is to be held at a constant value (which we shall refer to as $\|\delta y\|$). Note that $\rho_i \leq L$. We shall subsequently refer to Lindberg's LIPEST (FORTRAN version that was listed in [18] and Shampine's LIPBND subroutines which calculate a Lipschitz constant via the above technique. See [22] for further details about their procedures.

In the present algorithm we choose $y^{(0)} = y(a)$, but other things are done rather differently. First, we compute a maximum of three estimates (iterations), and we note that if the problem consists of only one equation, it is enough to form just one iteration. Next, for the differential equation $y' = f(x, y)$ defined on $[a, b]$, the formula

$$\tilde{y} = y(a) + \Delta x f(a, y(a)),$$

with $\operatorname{sgn}(\Delta x) = \operatorname{sgn}(b-a)$, advances an accurate approximation to the true solution at $a+\Delta x$ for all sufficiently small Δx. We think it is important to choose approximations of y, to be used in the difference formula, which are consistent with local integral curve behavior. Thus we shall insist that all perturbation vectors δy have components with the proper signs,

$$\operatorname{sgn}(\delta y_k) = \operatorname{sgn}\{(b-a) f_k(a, y(a))\}.$$

The question of what to do when one of the initial slopes vanishes will be addressed shortly. In rough terms, we shall refer to this idea as making perturbation increments in the directions dictated by the differential equations.

A couple of examples will illustrate the practical benefits of doing this. Let us consider the ideal relay equation [6]

$$y''(x) + y(x) + \operatorname{sgn}(y(x)) + 3 \sin 2x = 0,$$
$$y(0) = 0, \qquad y'(0) = 3$$

on an interval of say [0, 20]. The solution has jump discontinuities in its second derivative at multiples of $\frac{1}{2}\pi$ (in particular at $x = 0$). Thus only one sided Lipschitz constants are meaningful. In fact, if LIPBND is used we obtain an estimate of $L \approx 7.9 \cdot 10^6$ which reflects the fact that the process used positive and negative values for y. The same behavior occurs with LIPEST. Imposing the above constraints on perturbation directions the present algorithm obtains $L = 1$, which gives correct information on the local behavior of the differential equation that is pertinent to choosing a starting step size for the integration task.

Next, consider the equation

$$y'(x) = \frac{1}{1 + \sqrt{y(x)}}, \quad y(0) = 0$$

on an interval [0, 20]. A solution can be obtained as the positive root of $y + \frac{2}{3}y^{3/2} = x$ for each $x > 0$. Notice that $\partial f/\partial y$ is not defined at $y = 0$, and a Lipschitz constant does not exist in a strip containing $y = 0$. In this sense the problem is not well posed, and an ODE code should not be expected to treat such problems satisfactorily since underlying assumptions about the problem are not valid. Nevertheless, such problems are likely to be encountered, and it is desirable that the starting step size algorithm not cause program termination. To achieve this, the estimator for the Lipschitz constant should provide a 'reasonable sized' number, which is dependent upon the machine being used. When LIPBND is applied to this equation, a negative value for $y^{(2)}$ will be generated and a program stop occurs while attempting to evaluate the differential equation. A simple change of variable in the problem leads to the same difficulty when LIPEST is used. Imposing constraints on the perturbation directions leads to an estimate of $L \approx 1/\sqrt{\delta y} = u^{-3/16}$ which can be considered as acceptable.

Because we are primarily interested in finding a direction which exhibits a large change in f locally, we also think it is important to work with nonzero components in $y^{(0)} + \delta y^{(i)}$, similar to what Lindberg has done. However, we want to use the differential equations, in so far as is possible, to dictate the directions for the various components in the perturbation vector. Starting with $y^{(0)} = y(a)$ we compute a maximum of three iterations, forming $y^{(i)} = y^{(0)} + \delta y^{(i)}$ for $i = 1, 2, 3$. We define $\|\delta y\| = u^{3/8}\|y(a)\|$ (or just $u^{3/8}$ if this underflows to zero) and set

$$\delta y^{(1)} = \operatorname{sgn}(b-a) \frac{\|\delta y\|}{\|f(a, y(a))\|} f(a, y(a)).$$

Because we cannot have a null perturbation vector, if $\|f(a, y(a))\| = 0$ we arbitrarily set

$$\delta y^{(1)} = \operatorname{sgn}(b-a)\|\delta y\|/\|e\| e,$$

where e is the vector with all components equal to one. Otherwise, we do not force nonzero components at this level, preferring instead to utilize $f^{(1)} = f(a, y^{(1)})$ to aid in determining appropriate perturbation directions for those components that have zero slopes initially. Thus if $f_k^{(0)} = f_k(a, y(a)) = 0$, we then assign

$$\operatorname{sgn}(\delta y_k) = \operatorname{sgn}\{(b-a) f_k^{(1)}\}.$$

If $f_k^{(1)}$ is also zero, we have been unsuccessful in obtaining the kth component increment direction from the differential equation for the next iteration. In this case, we arbitrarily take

$$\operatorname{sgn}(\delta y_k) = \operatorname{sgn}(b-a).$$

Subsequent iterations are dealt with in a similar manner.

We then compute an estimate of the Lipschitz constant,

$$\rho_1 = \|f^{(1)} - f^{(0)}\|/\|\delta y\|,$$

and define the next iteration value $y^{(2)}$ from

$$\delta y_k^{(2)} = \operatorname{sgn}(\delta y_k)|f_k^{(1)} - f_k^{(0)}|/\rho_1.$$

In an attempt to obtain a better sampling of the local behavior of the differential equation, the next estimate is computed using a shifted value of the independent variable.

$$\rho_2 = \frac{\|f(a+\delta a, y^{(2)}) - f(a+\delta a, y^{(0)})\|}{\|\delta y\|},$$

where δa is the same value as described in the previous section. Notice that the second function evaluation here is already available from estimating $\|\partial f/\partial x\|$. The primary motivation was to move away a short distance from the present integral curve and to force computations with nonzero values which would have otherwise been zero. Because of using the shifted argument for the independent variable, we always perform at least two iterations (exactly two on problems with one equation). The third (and last) iteration value $y^{(3)}$ is defined by making nonzero perturbations of each component of $y(a)$ directly. That is,

$$\delta y_k^{(3)} = \operatorname{sgn}(\delta y_k)\frac{\|\delta y\|}{\|w\|} w_k$$

where

$$w_k = \begin{cases} y_k(a), & \text{if } y_k(a) \neq 0, \\ \|y(a)\|, & \text{otherwise.} \end{cases}$$

However, if all components of $y(a)$ are zero, we set $w_k = 1$. Finally, the last estimate for the Lipschitz constant is formed as

$$\rho_3 = \|f^{(3)} - f^{(0)}\|/\|\delta y\|,$$

where $f^{(3)} = f(a, y^{(3)})$. Then we choose max ρ_i as an approximation to L.

Let us now look at several examples which illustrate the usefulness of these ideas of introducing (forcing) nonzero perturbations and of making perturbations based solely on the sizes of the individual components of $y(a)$. In Table 1, we present comparisons of the spectral radius, spectral norm, and maximum norm of the Jacobian matrix J along with estimates of the Lipschitz constant computed both by LIPBND and the present algorithm. All quantities are computed at the initial data. The first example is the nonstiff problem D5 in [19], Newton's equations of motion for the simple two-body problem describing an elliptical orbit with eccentricity of 0.9. Writing as a first order system leads to four differential equations. The next two examples were taken from the stiff test set [8]. Problem E1 arose in control theory as a fourth order equation. Problem D2 is a widely used test example due to Robertson (this is actually a scaled version of the original) consisting of three differential equations. The next example is the original Robertson problem [20]. Problem D2 is the result of scaling the original solution components y_2 and y_3 by 10^4 and 10^2, respectively. Problem 2.8.10 was taken from [21, p. 156] and represents a chemistry model consisting of 12 differential equations. All four problems are nonlinear. We have used a CDC 6600 computer for all the numerical work in this paper and, for this machine, $u \doteq 7.1\ 10^{-15}$.

The estimate of the Lipschitz constant for the original Robertson problem [20] reflects the local behavior of the equations as you move away from the initial data. This is seen more clearly in Table 2 where the original and scaled problems are compared at $x = a = 0$, $x = 10^{-5}$ and $x = 10^{-4}$. Note that the Jacobian norms corresponding to the original problem vary more rapidly.

In summary, we expect our algorithm to pro-

Table 1
Comparison of spectral radius, spectral norm, maximum norm, and estimates of Lipschitz constants

Problem	$\rho(J)$	$\|J\|_2$	$\|J\|_\infty$	L	
				LIPBND	Present algorithm
D5, [19]	44.7	1414.2	2000	91.7	1000
E1, [8]	147.9	10^8	$1.04\ 10^8$	390	$1.04\ 10^8$
D2, [8]	0.04	400	400	0.04	400
Robertson, [20]	0.04	0.0566	0.04	1.83	148.5
2.8.10, [21]	1555.2	3468.6	4560.1	78.4	4560

Table 2
Comparison of local behavior of the Robertson problem – original and scaled versions (values are approximate)

	Original			Scaled		
	$\rho(J)$	$\|J\|_2$	$\|J\|_\infty$	$\rho(J)$	$\|J\|_2$	$\|J\|_\infty$
$x = 0$	0.04	0.06	0.04	0.04	400	400
$x = 10^{-5}$	24	33.9	24	24	401	424
$x = 10^{-4}$	239	338.1	239	239	466	643

duce a good result for a large lower bound on the local Lipschitz constant. Clearly we cannot guarantee an accurate approximation for $\|J\|$, but this is not necessary for our purposes. Any reasonable value reflecting the size of $\|J\|$ locally is perfectly acceptable and useful for the choice of initial step size. Additional information is provided in [22].

6. Starting step size algorithm

The algorithm computes estimates of $\|\partial f/\partial x\|$ and $\|\partial f/\partial y\|$, as described in the previous sections, along with an upper bound for $\|f\|$ near the initial point a. The latter quantity is obtained from all evaluations of f by the algorithm. With these, a bound on the norm of the second derivative is computed as

$$\|y''\| \approx \left\|\frac{\partial f}{\partial x}\right\| + \left\|\frac{\partial f}{\partial y}\right\| \|f\|.$$

A user-supplied (or code generated) error tolerance vector, with components $\tau_k > 0$, is assumed given. From this, we define a scalar tolerance parameter ε for use in the algorithm. We simply take $\varepsilon = 10^q$ where

$$q = \frac{1}{2}\left\{\frac{1}{n}\sum_{k=1}^{n} \log \tau_k + \min_k (\log \tau_k)\right\}.$$

This choice of ε 'fits between' the smallest and largest error tolerances(with a bias towards the smallest) and seems to be a reasonable compromise for problems having disparately scaled tolerance components.

Now, for $\|y''\| \neq 0$ we compute

$$|h| = \sqrt{2}\ \varepsilon^{1/p+1}/\sqrt{\|y''\|}\,.$$

However, if $\|y''\| = 0$ but $\|y'\| = \|f\| \neq 0$, we compute

$$|h| = \varepsilon^{1/p+1}/\|f\|,$$

which is just a version of the rule referred to earlier. If both $\|y'\| = \|y''\| = 0$, we rely on the integration interval length to provide information about the scale of the problem and compute

$$|h| = |b - a|\varepsilon^{1/p+1}.$$

We also believe in imposing further restrictions on the length of the starting step size for protection in circumstances when anomalies occur. One requirement is that

$$|h| \leqslant 1/\left\|\frac{\partial f}{\partial y}\right\|.$$

This was motivated by several reasons. Numerical stability of explicit methods imposes a limitation that the products $h\lambda_i$, where the λ_i are eigenvalues of the Jacobian matrix $J = \partial f/\partial y$, lie inside the region of absolute stability of the method. Characteristically, this can be expressed in the form $|h|\,|\lambda_i| \leqslant C[\arg(\lambda_i)]$ where C is method dependent and represents the distance from the origin to the boundary of the stability region along the ray given by the argument of λ_i. Typically, C varies in the range from about 10^{-1} to 10. Because $|\lambda_i| \leqslant \|J\|$, the above requirement seems about right. Using simple iteration with implicit methods imposes a similar constraint, viz., that $c|h|\,\|J\| < 1$ is a sufficient condition for convergence of the iterative process. Here c is a constant, generally of order one, which depends on the method. If $\|J\|$ is large but $\|f\|$ is small, the above requirement will determine the starting step size, which in some cases might be smaller than necessary. This could occur for example with stiff equations in which the initial conditions lie on the smooth, slowly changing part of the solution trajectory. Then the requested accuracy over one step is easy to achieve

without such a severe step size restriction. However, this situation does not appear to occur that often in practice. Moreover, we consider this to be a safety requirement which further ensures that the initial step taken be on scale of the problem.

Another restriction which we impose is that the starting step size length not be larger than the initial integration interval, $|h| \leqslant |b-a|$, unless b was chosen too close to a (relative to the precision of the machine). For this we decided to insist on $|h| \geqslant 100u|a|$. Next, if the value of $|h|$ obtained by the above requirements underflows to zero, we reset it to $|h| = u|b|$. Lastly, we define $h = |h|\text{sgn}(b-a)$.

So far we have not mentioned any particular vector norm except in some illustrations. This is principally because we have not seen any experimental evidence which would lead us to believe that one is preferable to another. However, Shampine [11] advances an argument in favor of the Euclidean vector norm for the scheme he uses. Our personal preference has always been the maximum vector norm and this is what we use in the subroutine. Any other vector norm could easily be used instead because we have designed the algorithm with this possibility in mind. The user of this algorithm merely has to replace the subprogram module which computes the maximum norm by another which computes the norm of choice. It is preferable to use the same vector norm that the ODE solver uses, but again we have not found this to be crucial.

7. Numerical experiments

We have performed numerous experiments with the starting step size algorithm. However, it is rather difficult to summarize the performance behavior with sets of statistics gleaned from any given test bed of problems. Nevertheless, we shall attempt to provide the reader with enough useful details so that he can judge the expected performance of this algorithm in an initial value code environment. We have studied problems from the nonstiff and stiff test sets [19,8] as well as many others derived from a number of sources. The experiments were conducted with the initial value codes from DEPAC [1]. These are DERKF which implements an imbedded fourth (fifth) order Runge–Kutta scheme, DEABM which implements a variable order (one through twelve) Adams method, and DEBDF which implements the variable order (one through five) backward differentiation formulas. Actually, these codes represent adapted versions of the well-known codes RKF45, ODE, and LSODE, respectively. Modifications were performed to make the codes conform to the DEPAC design, and the starting step size algorithm was inserted in all three.

We shall use the notation h_s to denote the starting step size obtained from the algorithm. Also, h_{opt} will denote the 'optimal' choice of step size according to the criterion: h_{opt} is the largest step size for which the estimate of the error in the solution approximation advanced over this step satisfies the requested error tolerance. Over a rather large collection of problems and a wide range of tolerances, we generally observed that for DERKF, $\frac{1}{5}|h_{opt}| \leqslant |h_s| \leqslant |h_{opt}|$. Rarely was the initial step found to be too large, and when it occurred, it was by only a small factor (2–5). In some instances ($\|J\|$ very large, as with quite stiff equations), h_s was reported to be a very small multiple of h_{opt}. This could be considered misleading since a closer examination of such problems revealed that in subsequent steps taken (within the first 20 steps being monitored), step rejections occurred with values in the range from two to ten times bigger than h_s. This is typical when the step size is limited because of stability considerations. h_{opt} is governed by accuracy estimates, so might well be considerably larger in such circumstances. However, we believe it is preferable to start off with h_s, which ensures that the procedure begins on scale of the problem.

With DEABM and DEBDF it was more often the case that $\frac{1}{2}|h_{opt}| \leqslant |h_s| \leqslant |h_{opt}|$. These better results are to be expected because DEABM and DEBDF start off with a first order method whereas DERKF starts at fourth order. Recall that the starting step size h_s is based on estimating second order terms, so we would anticipate getting the best results when used in conjunction with a first order method. On the other hand, there was a slightly greater tendency with these codes of having an initial step size larger than h_{opt}, again by only relatively small factors.

In order to present some more detailed comparisons with the use of the starting step size algorithm, we extracted small subsets of 14 nonstiff problems and 15 stiff problems which we

Table 3
Comparing use of the starting step size h_s in DERKF with using multiples of h_s and with RKF45. The number of derivative evaluations required are shown when using h_s on a set of fourteen nonstiff problems

Error tolerances	DERKF					RKF45
	$h_s/100$	$h_s/10$	h_s	$10h_s$	$100h_s$	
10^{-2} [a]	+173	+68	1074	+2	+480	+20
10^{-4}	+213	+88	4344	+2	+71	−49
10^{-6}	+253	+114	9625	+49	+112	−37
10^{-8}	+261	+124	20802	−9	+60	−57

[a] Five of the problems were not included for this tolerance.

considered to be sufficiently demanding or interesting. In Table 3, we show results for the Runge–Kutta procedure applied to the set of nonstiff problems for four tolerances. Statistics from five of the problems were excluded at the crudest tolerance because incorrect integral curves were ultimately followed by at least one of the procedures. The (total) numbers of derivative evaluations required by DERKF using h_s are given in the center column. Comparisons with RKF45 and with using multiples of h_s for starting step sizes are presented by showing the differences in the numbers of evaluations. The plus sign indicates the additional number required, meaning that it is less efficient than when using h_s. Minus indicates fewer evaluations required. Similar results are given for DEABM and ODE in Table 4. Notice that the sensitivity in numbers of derivative evaluations is not as large with the Runge–Kutta scheme as with the Adams method. Also, the biggest savings, percentage-wise, (due to using h_s) occurs at the crudest tolerances with the Runge–Kutta method while it occurs at the more stringent tolerances with the Adams method. The starting step sizes produced in both RKF45 and ODE are generally quite satisfactory for these particular problems. The advantage seen with RKF45 is about equal to the additional cost incurred with the starting step size algorithm – 33 evaluations for the problems at the crudest error tolerance and 53 evaluations at each of the more stringent tolerances. In Table 5, we show results from a set of stiff problems when using DEBDF and LSODE. Here we found a somewhat more erratic pattern emerging.

These results demonstrate that the starting step size algorithm produces initial step sizes which are 'roughly optimal' with the methods of DEPAC. A number of problems which caused difficulties with the initial step selection procedures in the predecessor codes are now being solved reliably. The cost of using the algorithm is minimal and most likely will be compensated for. Only four additional derivative evaluations are performed (three when the problem is a single equation). The overhead cost has been measured to be equivalent to about 2.5 times the cost of executing the FEHL

Table 4
Comparing use of the starting step size h_s in DEABM with using multiples of h_s and with ODE (STEP). The number of derivative evaluations required are shown when using h_s on a set of fourteen nonstiff problems

Error tolerances	DEABM					ODE (STEP)
	$h_s/100$	$h_s/10$	h_s	$10h_s$	$100h_s$	
10^{-2} [a]	−13	−58	1021	−17	0	−88
10^{-4}	+122	+25	3188	+116	+120	−27
10^{-6}	+178	+156	4821	+318	+435	+286
10^{-8}	+421	+220	6972	+391	+488	+172

[a] Five of the problems were not included for this tolerance.

Table 5
Comparing use of the starting step size h_s in DEBDF with using multiples of h_s and with LSODE. The number of derivative evaluations required are shown when using h_s on a set of fifteen stiff problems

Error tolerances	DEBDF					LSODE
	$h_s/100$	$h_s/10$	h_s	$10h_s$	$100h_s$	
10^{-2}	+64	+38	1681	+168	+107	+143
10^{-4}	+97	+87	3064	+100	+71	+155
10^{-6}	−11	+25	3901	+33	−24	+137

subroutine (excluding derivative evaluation costs) in RKF45. This routine performs the basic task of advancing a solution approximation over a single step via the Fehlberg–Runge–Kutta six stage explicit process.

8. Summary

We have discussed in detail an algorithm for computing a starting step size to be used by an initial value method in solving ordinary differential equations. We believe that it is quite a reasonable thing to do, and it has been demonstrated to be effective when used with the codes of DEPAC [1]. A transportable subroutine HSTART, written in FORTRAN, is provided in the appendix.

Acknowledgment

The author gratefully acknowledges the helpful benefits arising from frequent discussions and collaborative efforts with his colleague at Sandia, L.F. Shampine.

References

[1] L.F. Shampine and H.A. Watts, DEPAC – Design of a user oriented package of ODE solvers, SAND79-2374, Sandia National Laboratories, Albuquerque, NM, 1980.

[2] A.E. Sedgwick, An effective variable order variable step Adams method, Dept. of Computer Science Rept. 53, University of Toronto, Toronto, Canada, 1973.

[3] L.F. Shampine and M.K. Gordon, Some numerical experiments with DIFSUB, *SIGNUM Newsletter* **7** (1972) 24–26.

[4] L.F. Shampine and H.A. Watts, The art of writing a Runge–Kutta code, Part II, *Appl. Math. Comput.* **5** (1979) 93–121.

[5] G.E. Forsythe, M.A. Malcolm and C.B. Moler, *Computer Methods for Mathematical Computations* (Prentice-Hall, Englewood Cliffs, NJ, 1977).

[6] L.F. Shampine and M.K. Gordon, *Computer Solution of Ordinary Differential Equations: The Initial Value Problem* (Freeman, San Francisco, CA, 1975).

[7] T.E. Hull, W.H. Enright and K.R. Jackson, User's guide for DVERK – a subroutine for solving nonstiff ODEs, Dept. of Computer Science Rept. 100, University of Toronto, Toronto, Canada, 1976.

[8] W.H. Enright, T.E. Hull and B. Lindberg, Comparing numerical methods for stiff systems of ODEs, *BIT* **15** (1975) 10–48.

[9] L.F. Shampine, Starting an ODE solver, SAND77-1023, Sandia National Laboratories, Albuquerque, NM, 1977.

[10] L.F. Shampine, Limiting precision in differential equation solvers. II: Sources of trouble and starting a code, *Math. Comp.* **32** (1978) 1115–1122.

[11] L.F. Shampine, Lipschitz constants and robust ODE codes, in: J.T. Oden, Ed., *Computational Methods in Nonlinear Mechanics* (North-Holland, Amsterdam, 1980) 427–449.

[12] B. Lindberg, IMPEX 2 a procedure for solution of systems of stiff differential equations, Dept. of Information Processing Rept. TRITA-NA-7303, Royal Institute of Technology, Stockholm, Sweden, 1973.

[13] C.W. Gear, Runge–Kutta starters for multistep methods, *ACM Trans. Math. Software* **6** (1980) 263–279.

[14] C.W. Gear, Method and initial stepsize selection in multistep ODE solvers, Dept. of Computer Science Rept. UIUCDCS-R-80-1006, University of Illinois, Urbana, IL, 1980.

[15] A.R. Curtis and J.K. Reid, The choice of step length when using differences to approximate Jacobian matrices, *J. Inst. Math. Appl.* **13** (1974) 121–126.

[16] R.S. Stepleman and N.D. Winarsky, Adaptive numerical differentiation, *Math. Comp.* **33** (1979) 1257–1264.

[17] J. Oliver, An algorithm for numerical differentiation of a function of one real variable, *J. Comp. Appl. Math.* **6** (1980) 145–160.

[18] J. Oppelstrup, DELAY 1 a program for integrating systems of delay-differential equations, Dept. of Information Processing Rept. TRITA-NA-7311, Royal Institute of Technology, Stockholm, Sweden, 1973.

[19] T.E. Hull, W.H. Enright, B.M. Fellen and A.E. Sedgwick, Comparing numerical methods for ordinary differential equations, *SIAM J. Numer. Anal.* **9** (1972) 603–637.

[20] H.H. Robertson, The solution of a set of reaction rate equations, in: J. Walsh, Ed., *Numerical Analysis: An Introduction* (Academic Press, London, 1966).

[21] P.J. van der Houwen, *Construction of Integration Formulas for Initial Value Problems* (North-Holland, Amsterdam, 1977).

[22] H.A. Watts, Starting step size for an ODE solver, SAND80-1734, Sandia National Laboratories, Albuquerque, NM, 1982.

Subroutine HSTART

```
      SUBROUTINE HSTART(F,NEQ,A,B,Y,YPRIME,ETOL,MORDER,SMALL,BIG,
     1                                      SPY,PV,YP,SF,RPAR,IPAR,H)
C
C*********************************************************************
C  ABSTRACT
C
C     SUBROUTINE HSTART COMPUTES A STARTING STEP SIZE TO BE USED BY AN
C     INITIAL VALUE METHOD IN SOLVING ORDINARY DIFFERENTIAL EQUATIONS.
C     IT IS BASED ON AN ESTIMATE OF THE LOCAL LIPSCHITZ CONSTANT FOR THE
C     DIFFERENTIAL EQUATION    (LOWER BOUND ON A NORM OF THE JACOBIAN) ,
C     A BOUND ON THE DIFFERENTIAL EQUATION  (FIRST DERIVATIVE) , AND
C     A BOUND ON THE PARTIAL DERIVATIVE OF THE EQUATION WITH RESPECT TO
C     THE INDEPENDENT VARIABLE.
C     (ALL APPROXIMATED NEAR THE INITIAL POINT A)
C
C     SUBROUTINE HSTART USES A FUNCTION SUBPROGRAM VNORM FOR COMPUTING
C     A VECTOR NORM. THE MAXIMUM NORM IS PRESENTLY UTILIZED THOUGH IT
C     CAN EASILY BE REPLACED BY ANY OTHER VECTOR NORM. IT IS PRESUMED
C     THAT ANY REPLACEMENT NORM ROUTINE WOULD BE CAREFULLY CODED TO
C     PREVENT UNNECESSARY UNDERFLOWS OR OVERFLOWS FROM OCCURRING, AND
C     ALSO, WOULD NOT ALTER THE VECTOR OR NUMBER OF COMPONENTS.
C
C*********************************************************************
C  ON INPUT YOU MUST PROVIDE THE FOLLOWING
C
C       F -- THIS IS A SUBROUTINE OF THE FORM
C                               F(X,U,UPRIME,RPAR,IPAR)
C             WHICH DEFINES THE SYSTEM OF FIRST ORDER DIFFERENTIAL
C             EQUATIONS TO BE SOLVED. FOR THE GIVEN VALUES OF X AND THE
C             VECTOR  U(*)=(U(1),U(2),...,U(NEQ)) , THE SUBROUTINE MUST
C             EVALUATE THE NEQ COMPONENTS OF THE SYSTEM OF DIFFERENTIAL
C             EQUATIONS  DU/DX=F(X,U)  AND STORE THE DERIVATIVES IN THE
C             ARRAY UPRIME(*), THAT IS,  UPRIME(I) = * DU(I)/DX *  FOR
C             EQUATIONS I=1,...,NEQ.
C
C             SUBROUTINE F MUST NOT ALTER X OR U(*). YOU MUST DECLARE
C             THE NAME F IN AN EXTERNAL STATEMENT IN YOUR PROGRAM THAT
C             CALLS HSTART. YOU MUST DIMENSION U AND UPRIME IN F.
C
C             RPAR AND IPAR ARE REAL AND INTEGER PARAMETER ARRAYS WHICH
C             YOU CAN USE FOR COMMUNICATION BETWEEN YOUR PROGRAM AND
C             SUBROUTINE F. THEY ARE NOT USED OR ALTERED BY HSTART. IF
C             YOU DO NOT NEED RPAR OR IPAR, IGNORE THESE PARAMETERS BY
C             TREATING THEM AS DUMMY ARGUMENTS. IF YOU DO CHOOSE TO USE
C             THEM, DIMENSION THEM IN YOUR PROGRAM AND IN F AS ARRAYS
C             OF APPROPRIATE LENGTH.
C
C       NEQ -- THIS IS THE NUMBER OF (FIRST ORDER) DIFFERENTIAL EQUATIONS
C              TO BE INTEGRATED.
C
C       A -- THIS IS THE INITIAL POINT OF INTEGRATION.
C
C       B -- THIS IS A VALUE OF THE INDEPENDENT VARIABLE USED TO DEFINE
C             THE DIRECTION OF INTEGRATION. A REASONABLE CHOICE IS TO
```

```
C             SET  B  TO THE FIRST POINT AT WHICH A SOLUTION IS DESIRED.
C             YOU CAN ALSO USE  B, IF NECESSARY, TO RESTRICT THE LENGTH
C             OF THE FIRST INTEGRATION STEP BECAUSE THE ALGORITHM WILL
C             NOT COMPUTE A STARTING STEP LENGTH WHICH IS BIGGER THAN
C             ABS(B-A), UNLESS  B  HAS BEEN CHOSEN TOO CLOSE TO  A.
C             (IT IS PRESUMED THAT HSTART HAS BEEN CALLED WITH  B
C             DIFFERENT FROM  A  ON THE MACHINE BEING USED. ALSO SEE THE
C             DISCUSSION ABOUT THE PARAMETER  SMALL.)
C
C      Y(*) -- THIS IS THE VECTOR OF INITIAL VALUES OF THE NEQ SOLUTION
C             COMPONENTS AT THE INITIAL POINT  A.
C
C      YPRIME(*) -- THIS IS THE VECTOR OF DERIVATIVES OF THE NEQ
C             SOLUTION COMPONENTS AT THE INITIAL POINT  A.
C             (DEFINED BY THE DIFFERENTIAL EQUATIONS IN SUBROUTINE F)
C
C      ETOL -- THIS IS THE VECTOR OF ERROR TOLERANCES CORRESPONDING TO
C             THE NEQ SOLUTION COMPONENTS. IT IS ASSUMED THAT ALL
C             ELEMENTS ARE POSITIVE. FOLLOWING THE FIRST INTEGRATION
C             STEP, THE TOLERANCES ARE EXPECTED TO BE USED BY THE
C             INTEGRATOR IN AN ERROR TEST WHICH ROUGHLY REQUIRES THAT
C                        ABS(LOCAL ERROR)  .LE.  ETOL
C             FOR EACH VECTOR COMPONENT.
C
C      MORDER -- THIS IS THE ORDER OF THE FORMULA WHICH WILL BE USED BY
C             THE INITIAL VALUE METHOD FOR TAKING THE FIRST INTEGRATION
C             STEP.
C
C      SMALL -- THIS IS A SMALL POSITIVE MACHINE DEPENDENT CONSTANT
C             WHICH IS USED FOR PROTECTING AGAINST COMPUTATIONS WITH
C             NUMBERS WHICH ARE TOO SMALL RELATIVE TO THE PRECISION OF
C             FLOATING POINT ARITHMETIC.  SMALL  SHOULD BE SET TO
C             (APPROXIMATELY) THE SMALLEST POSITIVE REAL NUMBER SUCH
C             THAT  (1.+SMALL) .GT. 1.  ON THE MACHINE BEING USED. THE
C             QUANTITY  SMALL**(3/8)  IS USED IN COMPUTING INCREMENTS OF
C             VARIABLES FOR APPROXIMATING DERIVATIVES BY DIFFERENCES.
C             ALSO THE ALGORITHM WILL NOT COMPUTE A STARTING STEP LENGTH
C             WHICH IS SMALLER THAN  100*SMALL*ABS(A).
C
C      BIG -- THIS IS A LARGE POSITIVE MACHINE DEPENDENT CONSTANT WHICH
C             IS USED FOR PREVENTING MACHINE OVERFLOWS. A REASONABLE
C             CHOICE IS TO SET BIG TO (APPROXIMATELY) THE SQUARE ROOT OF
C             THE LARGEST REAL NUMBER WHICH CAN BE HELD IN THE MACHINE.
C
C      SPY(*),PV(*),YP(*),SF(*) -- THESE ARE REAL WORK ARRAYS OF LENGTH
C             NEQ WHICH PROVIDE THE ROUTINE WITH NEEDED STORAGE SPACE.
C
C      RPAR,IPAR -- THESE ARE PARAMETER ARRAYS, OF REAL AND INTEGER
C             TYPE, RESPECTIVELY, WHICH CAN BE USED FOR COMMUNICATION
C             BETWEEN YOUR PROGRAM AND THE F SUBROUTINE. THEY ARE NOT
C             USED OR ALTERED BY HSTART.
C
C**********************************************************************
C  ON OUTPUT  (AFTER THE RETURN FROM HSTART),
C
C      H -- IS AN APPROPRIATE STARTING STEP SIZE TO BE ATTEMPTED BY THE
C             DIFFERENTIAL EQUATION METHOD.
C
C           ALL PARAMETERS IN THE CALL LIST REMAIN UNCHANGED EXCEPT FOR
C           THE WORKING ARRAYS SPY(*),PV(*),YP(*), AND SF(*).
C
C**********************************************************************
C   REFERENCE
C
```

```
C   H.A. WATTS, *STARTING STEP SIZE FOR AN ODE SOLVER*, SAND80-1734,
C   SANDIA NATIONAL LABORATORIES, 1981.
C
C***********************************************************************
C
      DIMENSION Y(NEQ),YPRIME(NEQ),ETOL(NEQ),
     1          SPY(NEQ),PV(NEQ),YP(NEQ),SF(NEQ)        ,RPAR(1),IPAR(1)
      EXTERNAL F
C
C.....................................................................
C
      DX=B-A
      ABSDX=ABS(DX)
      RELPER=SMALL**0.375
C
C.....................................................................
C
C     COMPUTE AN APPROXIMATE BOUND (DFDXB) ON THE PARTIAL
C     DERIVATIVE OF THE EQUATION WITH RESPECT TO THE
C     INDEPENDENT VARIABLE. PROTECT AGAINST AN OVERFLOW.
C     ALSO COMPUTE A BOUND (FBND) ON THE FIRST DERIVATIVE LOCALLY.
C
      DA=SIGN(AMAX1(AMIN1(RELPER*ABS(A),ABSDX),100.*SMALL*ABS(A)),DX)
      IF (DA .EQ. 0.) DA=RELPER*DX
      CALL F(A+DA,Y,SF,RPAR,IPAR)
      DO 10 J=1,NEQ
   10   YP(J)=SF(J)-YPRIME(J)
      DELF=VNORM(YP,NEQ)
      DFDXB=BIG
      IF (DELF .LT. BIG*ABS(DA)) DFDXB=DELF/ABS(DA)
      FBND=VNORM(SF,NEQ)
C
C.....................................................................
C
C     COMPUTE AN ESTIMATE (DFDUB) OF THE LOCAL LIPSCHITZ CONSTANT FOR
C     THE SYSTEM OF DIFFERENTIAL EQUATIONS. THIS ALSO REPRESENTS AN
C     ESTIMATE OF THE NORM OF THE JACOBIAN LOCALLY.
C     THREE ITERATIONS (TWO WHEN NEQ=1) ARE USED TO ESTIMATE THE
C     LIPSCHITZ CONSTANT BY NUMERICAL DIFFERENCES. THE FIRST
C     PERTURBATION VECTOR IS BASED ON THE INITIAL DERIVATIVES AND
C     DIRECTION OF INTEGRATION. THE SECOND PERTURBATION VECTOR IS
C     FORMED USING ANOTHER EVALUATION OF THE DIFFERENTIAL EQUATION.
C     THE THIRD PERTURBATION VECTOR IS FORMED USING PERTURBATIONS BASED
C     ONLY ON THE INITIAL VALUES. COMPONENTS THAT ARE ZERO ARE ALWAYS
C     CHANGED TO NON-ZERO VALUES (EXCEPT ON THE FIRST ITERATION). WHEN
C     INFORMATION IS AVAILABLE, CARE IS TAKEN TO ENSURE THAT COMPONENTS
C     OF THE PERTURBATION VECTOR HAVE SIGNS WHICH ARE CONSISTENT WITH
C     THE SLOPES OF LOCAL SOLUTION CURVES.
C     ALSO CHOOSE THE LARGEST BOUND (FBND) FOR THE FIRST DERIVATIVE.
C
C                       PERTURBATION VECTOR SIZE IS HELD CONSTANT FOR
C                       ALL ITERATIONS. COMPUTE THIS CHANGE FROM THE
C                            SIZE OF THE VECTOR OF INITIAL VALUES.
      DELY=RELPER*VNORM(Y,NEQ)
      IF (DELY .EQ. 0.) DELY=RELPER
      DELY=SIGN(DELY,DX)
      DELF=VNORM(YPRIME,NEQ)
      FBND=AMAX1(FBND,DELF)
      IF (DELF .EQ. 0.) GO TO 30
C                       USE INITIAL DERIVATIVES FOR FIRST PERTURBATION
      DO 20 J=1,NEQ
        SPY(J)=YPRIME(J)
   20   YP(J)=YPRIME(J)
      GO TO 50
```

```
C                       CANNOT HAVE A NULL PERTURBATION VECTOR
   30 DO 40 J=1,NEQ
        SPY(J)=0.
   40   YP(J)=1.
      DELF=VNORM(YP,NEQ)
C
   50 DFDUB=0.
      LK=MIN0(NEQ+1,3)
      DO 140 K=1,LK
C                       DEFINE PERTURBED VECTOR OF INITIAL VALUES
        DO 60 J=1,NEQ
   60     PV(J)=Y(J)+DELY*(YP(J)/DELF)
        IF (K .EQ. 2) GO TO 80
C                       EVALUATE DERIVATIVES ASSOCIATED WITH PERTURBED
C                       VECTOR  AND  COMPUTE CORRESPONDING DIFFERENCES
        CALL F(A,PV,YP,RPAR,IPAR)
        DO 70 J=1,NEQ
   70     PV(J)=YP(J)-YPRIME(J)
        GO TO 100
C                       USE A SHIFTED VALUE OF THE INDEPENDENT VARIABLE
C                                             IN COMPUTING ONE ESTIMATE
   80   CALL F(A+DA,PV,YP,RPAR,IPAR)
        DO 90 J=1,NEQ
   90     PV(J)=YP(J)-SF(J)
C                       CHOOSE LARGEST BOUNDS ON THE FIRST DERIVATIVE
C                                    AND A LOCAL LIPSCHITZ CONSTANT
  100   FBND=AMAX1(FBND,VNORM(YP,NEQ))
        DELF=VNORM(PV,NEQ)
        IF (DELF .GE. BIG*ABS(DELY)) GO TO 150
        DFDUB=AMAX1(DFDUB,DELF/ABS(DELY))
        IF (K .EQ. LK) GO TO 160
C                       CHOOSE NEXT PERTURBATION VECTOR
        IF (DELF .EQ. 0.) DELF=1.
        DO 130 J=1,NEQ
          IF (K .EQ. 2) GO TO 110
          DY=ABS(PV(J))
          IF (DY .EQ. 0.) DY=DELF
          GO TO 120
  110     DY=Y(J)
          IF (DY .EQ. 0.) DY=DELY/RELPER
  120     IF (SPY(J) .EQ. 0.) SPY(J)=YP(J)
          IF (SPY(J) .NE. 0.) DY=SIGN(DY,SPY(J))
  130     YP(J)=DY
  140   DELF=VNORM(YP,NEQ)
C
C                       PROTECT AGAINST AN OVERFLOW
  150 DFDUB=BIG
C
C.....................................................................
C
C     COMPUTE A BOUND (YDPB) ON THE NORM OF THE SECOND DERIVATIVE
C
  160 YDPB=DFDXB+DFDUB*FBND
C
C.....................................................................
C
C     DEFINE THE TOLERANCE PARAMETER UPON WHICH THE STARTING STEP SIZE
C     IS TO BE BASED.  A VALUE IN THE MIDDLE OF THE ERROR TOLERANCE
C     RANGE IS SELECTED.
C
      TOLMIN=BIG
      TOLSUM=0.
      DO 170 K=1,NEQ
        TOLEXP=ALOG10(ETOL(K))
        TOLMIN=AMIN1(TOLMIN,TOLEXP)
```

```
  170   TOLSUM=TOLSUM+TOLEXP
        TOLP=10.**(0.5*(TOLSUM/FLOAT(NEQ)+TOLMIN)/FLOAT(MORDER+1))
C
C.....................................................................
C
C     COMPUTE A STARTING STEP SIZE BASED ON THE ABOVE FIRST AND SECOND
C     DERIVATIVE INFORMATION
C
C                       RESTRICT THE STEP LENGTH TO BE NOT BIGGER THAN
C                       ABS(B-A).   (UNLESS  B  IS TOO CLOSE TO  A)
      H=ABSDX
C
      IF (YDPB .NE. 0.  .OR.  FBND .NE. 0.) GO TO 180
C
C                       BOTH FIRST DERIVATIVE TERM (FBND) AND SECOND
C                                 DERIVATIVE TERM (YDPB) ARE ZERO
      IF (TOLP .LT. 1.) H=ABSDX*TOLP
      GO TO 200
C
  180 IF (YDPB .NE. 0.) GO TO 190
C
C                       ONLY SECOND DERIVATIVE TERM (YDPB) IS ZERO
      IF (TOLP .LT. FBND*ABSDX) H=TOLP/FBND
      GO TO 200
C
C                       SECOND DERIVATIVE TERM (YDPB) IS NON-ZERO
  190 SRYDPB=SQRT(0.5*YDPB)
      IF (TOLP .LT. SRYDPB*ABSDX) H=TOLP/SRYDPB
C
C                       FURTHER RESTRICT THE STEP LENGTH TO BE NOT
C                                              BIGGER THAN  1/DFDUB
  200 IF (H*DFDUB .GT. 1.) H=1./DFDUB
C
C                       FINALLY, RESTRICT THE STEP LENGTH TO BE NOT
C                       SMALLER THAN  100*SMALL*ABS(A).  HOWEVER, IF
C                       A=0. AND THE COMPUTED H UNDERFLOWED TO ZERO,
C                       THE ALGORITHM RETURNS  SMALL*ABS(B)  FOR THE
C                                                   STEP LENGTH.
      H=AMAX1(H,100.*SMALL*ABS(A))
      IF (H .EQ. 0.) H=SMALL*ABS(B)
C
C                       NOW SET DIRECTION OF INTEGRATION
      H=SIGN(H,DX)
C
      RETURN
      END




      FUNCTION VNORM(V,NCOMP)
C
C     COMPUTE THE MAXIMUM NORM OF THE VECTOR V(*) OF LENGTH NCOMP AND
C     RETURN THE RESULT AS VNORM
C
      DIMENSION V(NCOMP)
      VNORM=0.
      DO 10 K=1,NCOMP
   10   VNORM=AMAX1(VNORM,ABS(V(K)))
      RETURN
      END
```